Module 2: Python Fundamentals

Slide-wise Handout of Technical Terms

Modern Computing and Python Programming (26ESCC106)

Text Book: Reema Thareja, *Python Programming using Problem Solving Approach*, Oxford University Press, 3rd Edition, 2024 (Sections 3.1, 3.2, 3.3, 3.5, 3.6, 3.7, 3.12, 3.16; supporting: 3.4, 3.8–3.11, Annexure 4)

Manjunath Prasad H. R., Assistant Professor, Department of AIML, MITE

How to use this handout. Each heading matches one slide of the Module 2 lecture deck (the number in the black box is the slide number shown at the bottom-right of the slide). Under each heading, the technical terms that appear on that slide are defined and explained in simple language. Small Python examples are given wherever trying the code makes the idea easier to understand — type them at the >>> prompt in IDLE. All examples are written for **Python 3**.

Contents

Part A: Course Overview Slides (Slides 1–19) **3**

Part B: Module 2 – Python Fundamentals (Slides 20–76) **3**

Slide 20: Module 2 – Python Fundamentals: What We Will Learn . . . 3
Slide 21: What Is Python? . . . 3
Slide 22: Features of Python (1/2) . . . 4
Slide 23: Features of Python (2/2) . . . 4
Slide 24: How Python Runs Your Program . . . 5
Slide 25: Limitations of Python . . . 5
Slide 26: History of Python . . . 5
Slide 27: Applications of Python . . . 6
Slide 28: The Future of Python . . . 6
Slide 29: Writing and Executing the First Python Program . . . 6
Slide 30: Writing Python Programs: Four Steps . . . 6
Slide 31: Structure of a Python Program . . . 7
Slide 32: Comments . . . 7
Slide 33: Indentation . . . 7
Slide 34: Physical Lines, Logical Lines, and Statements . . . 8
Slide 35: Literal Constants . . . 8
Slide 36: Simple Operations on Numbers . . . 8
Slide 37: String Literals . . . 9
Slide 38: Escape Sequences and Raw Strings . . . 9
Slide 39: Variables . . . 9
Slide 40: Identifiers . . . 10
Slide 41: Keywords (Reserved Words) . . . 10
Slide 42: Output with `print()` . . . 10
Slide 43: Formatted Output with `format()` . . . 10
Slide 44: Input with `input()` . . . 11

Slide 45: Reading Numbers from the User . . . 11
Slide 46: Data Types in Python . . . 11
Slide 47: Assigning and Reassigning Values . . . 12
Slide 48: Multiple Assignment . . . 12
Slide 49: Boolean Data Type . . . 12
Slide 50: Operators and Operands . . . 12
Slide 51: Arithmetic Operators . . . 13
Slide 52: Comparison (Relational) Operators . . . 13
Slide 53: Assignment and Shortcut (In-place) Operators . . . 13
Slide 54: Logical Operators . . . 13
Slide 55: Unary, Bitwise, and Shift Operators . . . 14
Slide 56: Membership and Identity Operators . . . 14
Slide 57: Expressions . . . 14
Slide 58: Operator Precedence . . . 15
Slide 59: Precedence and Associativity: Examples . . . 15
Slide 60: Why Do We Need Type Conversion? . . . 15
Slide 61: Type Conversion Functions . . . 15
Slide 62: Implicit vs. Explicit Conversion . . . 16
Slide 63: Bugs and Debugging . . . 16
Slide 64: Reading an Error Message (Traceback) . . . 17
Slide 65: Common Errors for Beginners . . . 17
Slide 66: Debugging Aids . . . 17
Slide 67: Debugging Using the Python IDLE Debugger . . . 17
Slide 68: Preconditions for Effective Debugging . . . 17
Slide 69: Lab 1: Student Details, Total, and Average . . . 18
Slide 70: Lab 2: Area and Circumference of a Circle . . . 18
Slide 71: Lab 3: Values and Data Types – Program . . . 18
Slide 72: Lab 3: Values and Data Types – Sample Output . . . 18
Slide 73: Lab 3: Why Not `bool(input())`? . . . 18
Slide 74: Lab 4: Arithmetic Operations on Two Numbers . . . 18
Slide 75: Summary . . . 19
Slide 76: Review Questions . . . 19

Part A: Course Overview Slides

Slides 1–19 (title, outline, course description, objectives, module topics, laboratory components, course outcomes, textbooks, tools, and thank you) are the same as in the Module 1 deck. Their technical terms are explained in Part A of the Module 1 handout.

Part B: Module 2 – Python Fundamentals

Slide 20 Module 2 – Python Fundamentals: What We Will Learn

Program

Definition: A set of instructions, written in a programming language, that tells the computer what to do step by step.
Description: Like a recipe: the computer follows each step exactly in order.

Programming Language

Definition: A formal language with fixed rules (syntax) used to write programs.
Description: Examples: Python, C, Java.

Syntax

Definition: The rules of a language that decide how statements must be written.
Description: If a rule is broken, Python reports a `SyntaxError`.

Slide 21 What Is Python?

High-Level Language

Definition: A programming language that is close to human language and hides the details of the computer hardware.
Description: Opposite: low-level languages such as assembly language, which are close to machine instructions.

Interpreted Language

Definition: A language whose programs are run by an **interpreter** statement by statement, without a separate compilation step.

Interactive

Definition: The programmer can type a statement and see its result immediately.

Object-Oriented

Definition: A style of programming in which data and the operations on it are grouped together into **objects**.
Description: In Python, everything — numbers, strings, lists — is an object.

Open Source

Definition: Software whose source code is freely available to read, change, and share.

Module (Library)

Definition: A file of ready-made Python code (functions and variables) that can be reused in other programs.
Description: Example: the `math` module provides `math.sqrt()` and `math.pi`.

Platform-Independent

Definition: A program that runs on different operating systems (Windows, Linux, macOS) without changes.

Slide 22 Features of Python (1/2)

Versatile

Definition: Useful for many different kinds of work.

Free and Open-Source Software (FOSS)

Definition: Software that anyone can use, read, change, and distribute free of cost.

Memory Management

Definition: Reserving and freeing computer memory for a program's data. Python does this automatically.

Portable

Definition: Able to run on many hardware platforms and operating systems in the same way.

Procedure-Oriented Programming

Definition: A style that builds a program around procedures or functions that are reusable pieces of code.

Slide 23 Features of Python (2/2)

Interpreter

Definition: A program that reads source code and runs it statement by statement.
Description: Python's interpreter is called `python` (or IDLE provides one).

Compiler

Definition: A program that translates the entire source code into machine code **before** it runs (as in C).

Dynamic

Definition: Decisions such as the type of a variable are made while the program runs; errors are reported at run time.

Extensible and Embeddable

Definition: Extensible: new modules (written in C/C++) can be added to Python. Embeddable: Python can be placed inside programs written in other languages.

Exception

Definition: An error that occurs while a program is running, such as dividing by zero. Python reports it and the program can handle it.

Robust

Definition: Strong and reliable: errors are reported clearly instead of crashing silently.

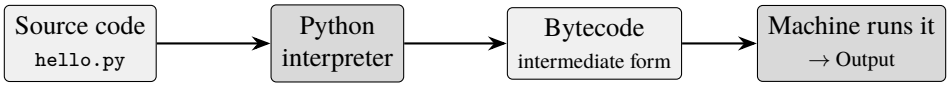
Multi-threaded

Definition: Able to run several parts (threads) of a program at the same time.

Garbage Collection

Definition: Automatically freeing the memory of objects that are no longer used by the program.
Description: This saves programmers from memory leaks.

Slide 24 How Python Runs Your Program



Source Code

Definition: The program as written by the programmer, saved in a `.py` file.

Bytecode

Definition: An intermediate, low-level form of the program that Python creates from the source code before running it.
Description: Bytecode makes Python portable: the same bytecode can run on any computer that has Python.

Run Time

Definition: The period during which a program is executing.

Compiling, Linking, and Loading

Definition: The steps needed in languages like C before a program can run: translating, joining with libraries, and placing it in memory. Python does not need these separately.

Slide 25 Limitations of Python

Executable File

Definition: A single file (such as `.exe` on Windows) that can run on its own without installing Python.
Description: It is difficult to package a large Python application into one executable file.

Parallel Processing

Definition: Running many calculations at the same time on several processors (studied in Module 1).

Documentation

Definition: Written material that explains how to use a language or software.

Slide 26 History of Python

Guido van Rossum

Definition: The Dutch programmer who created Python at CWI (the National Research Institute for Mathematics and Computer Science) in the Netherlands.

Version

Definition: A numbered release of software. Python 3 (released in 2008) is used in this course; Python 2 is no longer supported.

Monty Python's Flying Circus

Definition: A BBC comedy series from the 1970s from which Python got its name.

Slide 27 Applications of Python

Framework

Definition: A ready-made structure of code on which programmers build applications.
Description: Examples: Django and Flask for websites.

GUI (Graphical User Interface)

Definition: A visual interface with windows, buttons, and icons.
Description: Python toolkits for GUIs: wxPython, PyQt, PyGtk.

Scripting

Definition: Writing small programs (scripts) that automate tasks.

Prototyping

Definition: Building a quick, simple version of a product to test an idea before building the full product.

Slide 28 The Future of Python

Programming Paradigm

Definition: A style of programming, such as object-oriented, functional, or parallel programming.
Description: Python supports several paradigms.

Slide 29 Writing and Executing the First Python Program

IDLE (Integrated Development and Learning Environment)

Definition: The simple editor and shell that comes with Python, used to write and run programs.

Python Prompt (>>>)

Definition: The symbol shown by the Python shell when it is ready for the next statement.

Shell

Definition: The window where Python statements are typed and their results are shown immediately.

Interactive Mode vs. Script Mode

Definition: Interactive mode: statements typed one at a time at the prompt. Script mode: the whole program is written in a `.py` file and run together.

`print()` Function

Definition: A built-in function that displays values on the screen.

Slide 30 Writing Python Programs: Four Steps

Editor

Definition: A program used to type and edit source code (IDLE's editor, VS Code, Notepad++).

File Extension `.py`

Definition: The ending of a file name that marks it as a Python program.

Run Module (F5)

Definition: The IDLE command that runs the program in the current editor window.

Command Prompt

Definition: A text window where commands such as `python program.py` can be typed.

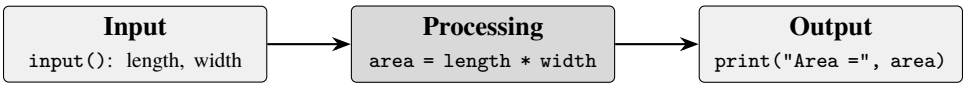
Slide 31 Structure of a Python Program

Statement

Definition: A single instruction that Python can execute, such as `area = length * width`.

Input, Processing, Output

Definition: The three parts of most programs: read data, calculate, and display the result.



Sequential Execution

Definition: Statements are executed one after another from top to bottom.

Slide 32 Comments

Comment

Definition: A note written in the program for humans; Python ignores it. A comment starts with `#` and continues to the end of the line.

Non-executable Statement

Definition: A line that is not run by the computer, such as a comment.

Slide 33 Indentation

Indentation

Definition: The spaces or tabs at the beginning of a line.
Description: In Python, indentation groups statements into a block.

Block

Definition: A group of statements that belong together and have the same indentation.
Description: Used with `if`, loops, and functions (Module 3).

`IndentationError`

Definition: The error Python reports when indentation is wrong, e.g., an unexpected space at the start of a line.

Whitespace

Definition: Characters that create blank space: spaces, tabs, and new lines.

Slide 34 Physical Lines, Logical Lines, and Statements

Physical Line

Definition: A line as we see it in the editor.

Logical Line

Definition: What Python treats as a single statement.
Description: Usually one physical line is one logical line.

Semicolon (;)

Definition: Separates two statements written on the same line. Avoid it; write one statement per line.

Slide 35 Literal Constants

Literal Constant

Definition: A fixed value written directly in a program, such as `7`, `3.9`, or `"Hello"`.

Integer

Definition: A whole number without a decimal point, such as `5` or `-12`. Python integers can be of any size.

Floating-Point Number (Float)

Definition: A number with a decimal point, such as `3.23`.

Scientific (E) Notation

Definition: A way of writing very large or small numbers using powers of 10: `91.5E-2` means $91.5 \times 10^{-2} = 0.915$.

Complex Number

Definition: A number with a real and an imaginary part, written in Python as `3 + 7j`.

Slide 36 Simple Operations on Numbers

Precision

Definition: The number of significant digits a number can hold. Python floats hold about 15–17 digits.

Loss of Precision

Definition: A small error because a float cannot store some values exactly: `0.1 + 0.2` gives `0.30000000000000004`.

Arithmetic Overflow

Definition: A result too large to be stored; Python shows `inf` (infinity) for floats.

Arithmetic Underflow

Definition: A result too small to be stored; Python shows `0.0`.

`ZeroDivisionError`

Definition: The error raised when a number is divided by zero.

`format()` Function

Definition: A built-in function that returns a value as a formatted string, e.g., `format(16/3, '.2f')` gives `'5.33'`.

Slide 37 String Literals

String

Definition: A group (sequence) of characters, written inside single, double, or triple quotes.

Triple-Quoted String

Definition: A string written within `'''…'''` or `"""…"""`; it can span several lines.

Immutable

Definition: Cannot be changed after it is created. Strings are immutable.

Concatenation

Definition: Joining two strings end to end, e.g., `"Good " + "Morning"` gives `"Good Morning"`.

Slide 38 Escape Sequences and Raw Strings

Escape Sequence

Definition: A backslash followed by a character that represents a special character: `\n` (new line), `\t` (tab), `\'` (single quote).

```
>>> print("Line 1\nLine 2")
Line 1
Line 2
>>> print('What\'s your name?')
What's your name?
```

Backslash (\)

Definition: The escape character in Python strings.

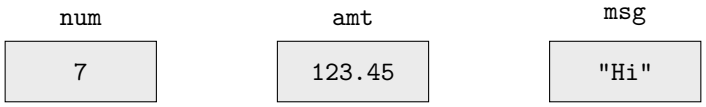
Raw String

Definition: A string prefixed with `r` or `R` in which escape sequences are not processed.
Description: Example: `r"C:\new"` prints `C:\new` exactly.

Slide 39 Variables

Variable

Definition: A named, reserved memory location that stores a value which can change while the program runs.



Each variable is a labelled box in memory: the name is the label, the value is inside.

Memory Location

Definition: A place in the computer's RAM where a value is stored.

Assignment

Definition: Storing a value in a variable using `=`, e.g., `num = 7`.

Slide 40 Identifiers

Identifier

Definition: A name given to a variable, function, class, module, or other object.

Case-Sensitive

Definition: Upper-case and lower-case letters are treated as different: `Total` and `total` are two different names.

Underscore (_)

Definition: A character allowed in identifiers, often used to join words: `total_marks`.

Slide 41 Keywords (Reserved Words)

Keyword (Reserved Word)

Definition: A word with a fixed meaning in Python that cannot be used as an identifier, e.g., `if`, `for`, `True`.
Description: Python 3.11 has 35 keywords.

`None`

Definition: A special keyword that means "no value".

`import`

Definition: A keyword used to bring a module into a program, e.g., `import keyword`.

Slide 42 Output with `print()`

Output

Definition: Information a program displays or sends out, usually on the screen.

Argument

Definition: A value given to a function inside its brackets, e.g., in `print("Hi", 5)`, `"Hi"` and `5` are arguments.

`sep` and `end`

Definition: Optional arguments of `print()`: `sep` is placed between values (default: a space); `end` is printed at the end (default: a new line).

Slide 43 Formatted Output with `format()`

Format Specifier

Definition: A short code that tells `format()` how to display a value: `'.2f'` (2 decimals), `'e'` (scientific), `','` (commas), `'<10'` (left-aligned in 10 spaces).

Alignment

Definition: Placing text at the left (<), right (>), or centre (^) of a given width.

Slide 44 Input with `input()`

Input

Definition: Data given to a program while it runs, usually typed by the user.

`input()` Function

Definition: A built-in function that displays a prompt, waits for the user to type, and returns the typed text **as a string**.

Prompt

Definition: The message shown to tell the user what to type, e.g., `"Enter your age: "`.

`raw_input()`

Definition: The Python 2 name of `input()`. It does not exist in Python 3.

Slide 45 Reading Numbers from the User

`int(input(…))`

Definition: Reads text from the user and converts it to an integer.

`float(input(…))`

Definition: Reads text from the user and converts it to a floating-point number.

```
>>> age = input("Age: ")        # user types 18
>>> age + 1
TypeError: can only concatenate str (not "int") to str
>>> int(age) + 1
19
```

Slide 46 Data Types in Python

Data Type

Definition: The kind of value a variable holds; it decides the operations allowed and the memory reserved.

`int`, `float`, `complex`, `str`, `bool`

Definition: The names of Python's basic data types: integer, floating-point number, complex number, string, Boolean.

`type()` Function

Definition: A built-in function that tells the data type of a value, e.g., `type(3.9)` gives `<class 'float'>`.

List, Tuple, Dictionary

Definition: Data types that hold collections of values. *(covered later, Module 4)*

Slide 47 Assigning and Reassigning Values

Assignment Operator (=)

Definition: Stores the value on the right side in the variable on the left side.
Description: It is not "equal to"; that is `==`.

Reassignment

Definition: Giving a variable a new value; the old value is replaced.

Dynamic Typing

Definition: A variable has no fixed type; it takes the type of the value currently stored in it.
Description: The same variable can hold `'Hello'`, then `100`, then `12.34`.

Slide 48 Multiple Assignment

Multiple Assignment

Definition: Assigning values to several variables in one statement: `a = b = 0` or `a, b = 3, 5`.

`del` Statement

Definition: Deletes a variable; using it afterwards gives a `NameError`.

`NameError`

Definition: The error raised when a variable name is used before a value is assigned to it (or after it is deleted).

Slide 49 Boolean Data Type

Boolean

Definition: A data type with only two values: `True` and `False`.
Description: Named after the mathematician George Boole.

Relational Expression

Definition: An expression that compares values and gives a Boolean result, e.g., `20 == 30` gives `False`.

Slide 50 Operators and Operands

Operator

Definition: A symbol that performs an operation, such as `+` or `*`.

Operand

Definition: A value on which an operator works. In `a + b`, `a` and `b` are operands.

Unary and Binary Operators

Definition: A unary operator works on one operand (`-b`); a binary operator works on two (`a + b`).

Slide 51 Arithmetic Operators

Modulus (%)

Definition: Gives the remainder of a division: `17 % 5` is 2.

Floor Division (//)

Definition: Gives the quotient rounded down to the nearest whole number: `17 // 5` is 3; `-19 // 5` is -4.

Exponentiation (**)

Definition: Raises a number to a power: `2 ** 3` is 8.

Quotient and Remainder

Definition: When 78 is divided by 5, the quotient is 15 (`78 // 5`) and the remainder is 3 (`78 % 5`).

Slide 52 Comparison (Relational) Operators

Comparison Operator

Definition: An operator that compares two values and returns `True` or `False`: `==`, `!=`, `>`, `<`, `>=`, `<=`.

Lexicographic Comparison

Definition: Comparing strings in dictionary (alphabetical) order: `"apple" < "banana"` is `True`.

Slide 53 Assignment and Shortcut (In-place) Operators

Shortcut (In-place / Compound) Operator

Definition: An operator that combines an arithmetic operation with assignment: `a += b` means `a = a + b`.
Description: Others: `-=`, `*=`, `/=`, `%=`, `//=`, `**=`.

Increment / Decrement

Definition: Increasing or decreasing a value by 1. Python has no `++` or `--`; write `count += 1`.

Slide 54 Logical Operators

Logical Operator

Definition: An operator that combines conditions: `and`, `or`, `not`.

Truth Table

Definition: A table that lists the result of a logical operation for every combination of inputs.

Short-Circuit Evaluation

Definition: Python stops evaluating a logical expression as soon as the result is known.
Description: In `False and X`, `X` is never evaluated.

Slide 55 Unary, Bitwise, and Shift Operators

Unary Minus

Definition: Changes the sign of a number: if `b = 10`, then `-b` is `-10`.

Bit and Binary Number

Definition: A bit is a single 0 or 1. Numbers are stored in binary: 10 is `1010`, 12 is `1100`.

Bitwise Operators (`&`, `|`, `^`, `~`)

Definition: Operators that work bit by bit on integers: AND, OR, XOR (exclusive OR), and NOT (complement).

Shift Operators (`<<`, `>>`)

Definition: Move all bits left or right. A left shift by 1 doubles the number; a right shift by 1 halves it.

XOR (Exclusive OR)

Definition: Gives 1 when exactly one of the two bits is 1.

Slide 56 Membership and Identity Operators

Membership Operators (`in`, `not in`)

Definition: Test whether a value is present in a sequence such as a string or list.

Sequence

Definition: An ordered collection of items, such as a string, list, or tuple.

Identity Operators (`is`, `is not`)

Definition: Test whether two names refer to the same object in memory.

`id()` Function

Definition: Returns a unique number that identifies an object in memory. `a is b` is `True` when `id(a) == id(b)`.

Slide 57 Expressions

Expression

Definition: A legal combination of operands and operators that produces a value, e.g., `A * B + C - 5`.

Infix, Prefix, Postfix

Definition: Positions of the operator: between the operands (`a + b`), before them (`+ a b`), or after them (`a b +`). Python uses infix.

Integral, Floating-Point, Relational, Logical, Bitwise, Assignment Expressions

Definition: Types of expressions based on the result they produce or the operators they use.

Slide 58 Operator Precedence

Operator Precedence

Definition: The priority of operators that decides which operation is done first in an expression.

BEDMAS

Definition: Brackets, Exponents, Division and Multiplication, Addition and Subtraction — the order used in mathematics and in Python.

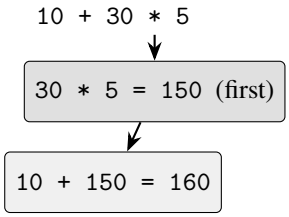
Parentheses

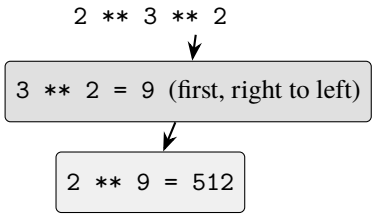
Definition: Brackets `( )` that force the operation inside them to be done first.

Slide 59 Precedence and Associativity: Examples

Associativity

Definition: The order (left to right or right to left) in which operators of the **same** precedence are evaluated.





Slide 60 Why Do We Need Type Conversion?

Type Conversion

Definition: Changing a value from one data type to another, e.g., the string `"25"` to the integer `25`.

`TypeError`

Definition: The error raised when an operation is used on the wrong type, e.g., `"Python" + 3.4`.

Slide 61 Type Conversion Functions

`int()`, `float()`, `str()`, `bool()`

Definition: Built-in functions that convert a value to an integer, float, string, or Boolean.

Truncation

Definition: Removing the decimal part without rounding: `int(2.90)` gives `2`.

Rounding

Definition: Changing a number to the nearest value: `round(2.90)` gives `3`; `round(89.567890, 2)` gives `89.57`.

`ord()` and `chr()`

Definition: `ord('A')` gives the character's number code `65`; `chr(66)` gives the character `'B'`.

Hexadecimal and Octal

Definition: Number systems with base 16 and base 8: `hex(100)` is `'0x64'`, `oct(100)` is `'0o144'`.

`ValueError`

Definition: The error raised when a value has the right type but an invalid content, e.g., `int("abc")`.

Slide 62 Implicit vs. Explicit Conversion

Implicit Conversion (Type Coercion)

Definition: Conversion done automatically by Python, e.g., in `21 + 2.1`, `21` becomes `21.0`.

Explicit Conversion (Type Casting)

Definition: Conversion done by the programmer using a function such as `int()` or `str()`.

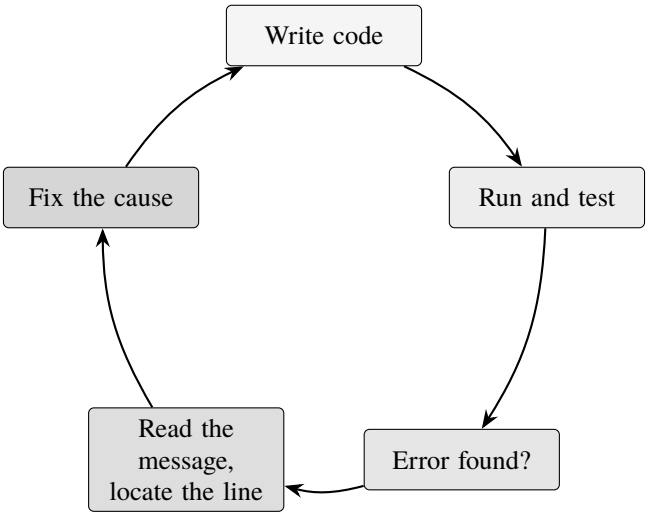
Slide 63 Bugs and Debugging

Bug

Definition: An error in a program that makes it fail or give wrong results.

Debugging

Definition: The process of finding and fixing bugs.



Syntax Error

Definition: An error in the way a statement is written; the program does not start.

Run-time Error (Exception)

Definition: An error that occurs while the program runs, stopping it in the middle.

Logical Error

Definition: An error in the logic: the program runs without any message but gives a wrong answer.

Testing

Definition: Running a program with different inputs to check whether it gives correct results.
Description: Testing shows that bugs exist; it cannot prove that a program is bug-free.

Slide 64 Reading an Error Message (Traceback)

Traceback

Definition: Python's error report, showing the file, the line number, the statement, and the type of error. Read it from the bottom up.

Error Type

Definition: The name of the error on the last line of a traceback, such as `TypeError` or `NameError`.

Slide 65 Common Errors for Beginners

Smart Quotes

Definition: Curly quotation marks (“ ”) produced by word processors and chat apps. Python accepts only straight quotes (`'` and `"`).

Initialize

Definition: Give a variable its first value before using it.

Slide 66 Debugging Aids

Print Debugging

Definition: Adding `print()` statements to see the values of variables while the program runs.

`assert` Statement

Definition: Checks a condition; if it is `False`, the program stops with an `AssertionError`.

Debugger

Definition: A tool that runs a program line by line so that the programmer can watch variables change.

Slide 67 Debugging Using the Python IDLE Debugger

Breakpoint

Definition: A marker on a line that makes the debugger pause there, so the programmer can examine variables.

Debug Control Window

Definition: IDLE's debugger window with buttons Go, Step, Over, Out, and Quit, and a Locals pane.

Stepping

Definition: Running a program one line at a time.

Locals Pane

Definition: The part of the Debug Control window that shows the current values and types of variables.

Slide 68 Preconditions for Effective Debugging

Algorithm

Definition: A step-by-step method for solving a problem.

Code Tracing

Definition: Following a program by hand, line by line, and writing down the value of each variable.

Peer Review

Definition: Having another person read your code to find errors you may have missed.

Test Case

Definition: A specific input together with its expected output, used to check a program.

Slide 69 Lab 1: Student Details, Total, and Average

Total and Average

Definition: Total is the sum of the marks; average is the total divided by the number of courses.

Formatted Display

Definition: Showing output neatly, with labels and aligned values, using `print()` and `format()`.

Slide 70 Lab 2: Area and Circumference of a Circle

Area and Circumference

Definition: Area of a circle $= \pi r^2$; circumference (perimeter) $= 2\pi r$, where r is the radius.

Constant (Named Constant)

Definition: A variable whose value is not meant to change, such as `PI = 3.14159`. By convention, its name is written in capital letters.

Slide 71 Lab 3: Values and Data Types – Program

`len()` Function

Definition: Returns the number of characters in a string: `len("Python")` is `6`.

String Repetition

Definition: Using `*` with a string and an integer repeats the string: `"Hi" * 3` gives `"HiHiHi"`.

Slide 72 Lab 3: Values and Data Types – Sample Output

No new terms. Note that `25 + 3.5` uses implicit conversion (Slide 62) and `True + 1` gives `2` (Slide 49).

Slide 73 Lab 3: Why Not `bool(input())`?

Truthiness

Definition: In Python, every value can be treated as `True` or `False`. Any non-empty string is `True`; the empty string `""` is `False`.
Description: So `bool("False")` is `True`.

Slide 74 Lab 4: Arithmetic Operations on Two Numbers

No new terms. All six operators were explained on Slide 51.

Slide 75 Summary

This slide revisits terms already defined. Quick recap:

Term	One-line meaning	Slide
Python	High-level, interpreted, interactive, object-oriented language	21–23
Comment, indentation	Notes for humans; spaces that group statements	32–33
Literal	A fixed value written in the program	35–38
Variable, identifier	Named memory location; the name given to it	39–40
Keyword	Reserved word that cannot be an identifier	41
`input()`, `print()`	Read text from the user; display values	42–45
Data type	Kind of value: `int`, `float`, `str`, `bool`, …	46–49
Operator, expression	Symbol that operates; combination giving a value	50–57
Precedence, associativity	Which operator first; order for equal precedence	58–59
Type conversion	Implicit (coercion) or explicit (casting)	60–62
Debugging	Finding and fixing bugs	63–68

Slide 76 Review Questions

No new terms. Pointers to the slides that answer each question:

Q.	Question	Refer to
1	Features of Python	Slides 21–24
2	History, applications, and limitations	Slides 25–28
3	Writing and executing a program; comments and indentation	Slides 29–34
4	Literal constants	Slides 35–38
5	Identifiers and their naming rules	Slides 39–41
6	`input()` and `print()`	Slides 42–45
7	Arithmetic, comparison, and logical operators	Slides 50–54
8	Operator precedence and associativity	Slides 58–59
9	Type conversion	Slides 60–62
10	Debugging, types of errors, and debugging aids	Slides 63–68